

JEPA Implementation Strategy for SNNJepa

Executive Summary

SNNJepa is a deliberately narrowed research workspace, hosted on GitHub ¹, whose stated purpose is to explore JEPA-style learning on top of a spiking substrate while preserving a working retina-classification benchmark for MNIST, EMNIST, and CIFAR-10. The docs are unusually clear about the intended direction: keep the spiking sensory frontend, keep the current baselines runnable as regression checks, and add JEPA-like mechanisms such as predictor/target branches, masked or partial-view prediction, temporal state alignment, and latent-state consistency without pivoting to ANN-to-SNN conversion or surrogate-gradient-first baselines. ²

The repository already has a substantial substrate that JEPA can reuse: a protected benchmark harness centered on `experiments/retina_classification.cpp`, retinal sensory adapters, generic spike-encoding abstractions, classification readouts, bilateral fusion, and a broader spiking runtime under `src/` and `include/`. What it does **not** have, in the current repo surface, is the thing JEPA fundamentally needs: an explicit context/target latent split, a predictor module, a masking sampler that creates nontrivial latent prediction tasks, and a loss-driven training loop for predictive latent alignment. The docs say this directly in roadmap form, and the file inventory corroborates it. ³

The single most important analytical conclusion is that the safest path is **JEPA-on-top-of-existing-spike-rollout**, not “replace the benchmark with a new learning stack.” That matches both the JEPA literature and the repo’s own warnings from `NEXT_STEPS.md` and `RabbitHoles.md`: the project has repeatedly lost time on downstream tuning, extra biological structure, and harness entanglement when upstream representation quality was weak or unverified. In other words, the right first JEPA milestone is not a smarter classifier. It is a better, instrumented latent code produced from the current spiking rollout, with explicit proof of mechanistic validity, stage-wise separability, and then a protected-benchmark win. ⁴

In practical terms, I recommend an incremental architecture in which the existing spiking retinal pipeline remains the sensory encoder, while new lightweight latent heads are added above stage-1 branch/hemisphere states: a context head, an EMA-updated target head, and a predictor head conditioned on masked fixation/branch positions. The first success criterion should be latent stability and separability on MNIST and EMNIST, using the current readout stack only as a probe. Only after that should the project spend time on replay, recurrent object-state JEPA variants, or CIFAR-scale promotion. That recommendation is strongly aligned with Yann LeCun ⁵’s JEPA framing and the Meta AI ⁶ I-JEPA design, where prediction happens in representation space, target encoders are stabilized by EMA, and masking is the core source of nontrivial predictive structure. ⁷

Repository Inventory and JEPA Mapping

The repo docs identify the retained benchmark executable and the intended extension points with unusual specificity. `ARCHITECTURE.md` says the active runtime surface is `experiments/`

`retina_classification.cpp`, using configs, dataset loaders, the retinal frontend, classification/fusion code, and the broader graph runtime. `README.md`, `REPO_LAYOUT.md`, and the top-level tree all confirm that the workspace is intentionally narrow: configs, docs, experiments, include, scripts, shaders, src, third_party, and top-level CMake, with no visible top-level `tests/` directory. ⁸

JEPA function	Best current repo surface	What is already there	Status	Evidence
Data pipeline and benchmark harness	<code>configs/</code> , <code>scripts/</code> , <code>experiments/retina_classification.cpp</code> , dataset loaders	Three retained baseline regimes; current benchmark executable; loaders for the retained visual domains	Implemented	<code>README.md</code> , <code>ARCHITECTURE.md</code> , <code>BASELINES.md</code> , <code>REPO_LAYOUT.md</code> ⁹
Spiking sensory encoder	<code>src/adapters/RetinaAdapter.cpp</code> , <code>include/snnfw/adapters/*</code> , <code>src/encoding/*</code> , <code>include/snnfw/encoding/*</code>	Retina frontend plus generic rate, temporal, and population encoding abstractions	Implemented	Active extension point named in architecture doc; adapter and encoding inventories; adapter README describes sensory adapters converting data into spike trains ¹⁰
Stage-1 state formation	<code>experiments/retina_classification.cpp</code>	Baseline path explicitly says each hemisphere builds stage-1 activation patterns	Implemented	<code>ARCHITECTURE.md</code> baseline path ¹¹

JEPA function	Best current repo surface	What is already there	Status	Evidence
Readout and late fusion	<code>src/classification/*</code> , <code>include/snnfw/classification/*</code> , bilateral fusion in benchmark path	Majority voting, weighted distance/similarity, hierarchical KNN, and corpus-callosum-style bilateral fusion surface	Implemented	Classification inventories and architecture/baseline docs ¹²
Replay / plasticity scaffolding	<code>src/learning/*</code> , <code>include/snnfw/learning/*</code> , benchmark-harness temporal rollout/replay extension points	Pattern-update strategies and docs that call out temporal rollout and replay logic as JEPA-relevant extension points	Partial	Learning inventories plus architecture doc ¹³
JEPA context state	No dedicated module yet; likely to sit above stage-1 branch/hemisphere patterns	Natural candidates exist in fixations, branches, hemispheres, and recurrent/object-state surfaces, but no explicit context-latent object exists	Missing as first-class module	<code>JEPA_ROADMAP.md</code> asks for separate context/predictor state; architecture doc names object-state and predictor-state storage as future extension points ¹⁴

JEPA function	Best current repo surface	What is already there	Status	Evidence
JEPA target state and target encoder	No dedicated target module in current tree	Planned explicitly; not present in directory structure	Missing	JEPA_ROADMAP.md and DEVELOPMENT.md both name explicit target latent states as first milestones 15
Predictor head	No dedicated predictor module in current tree	Planned explicitly; not present in directory structure	Missing	README.md , JEPA_ROADMAP.md , DEVELOPMENT.md 16
JEPA masking / partial observation	Repo direction exists, but no first-class JEPA masking pipeline is documented	The docs want masked-view or withheld-fixation targets; prior figure-ground / mask-style downstream variants are explicitly cautioned against as a repeated dead end	Missing as JEPA training primitive	README.md , JEPA_ROADMAP.md , NEXT_STEPS.md 17
Formal tests and CI gating	No visible top-level tests/ ; top-level CMake only adds src and experiments	Protected baselines and decision gates are documented, but repo-native automated tests are not visible in the retained surface	Partial at best	Root tree and CMake, plus documented experiment gates 18

The repo's own documentation strongly implies that JEPA work should enter as a **new representational layer above the existing spiking rollout**, not as a replacement for the retinal benchmark or a deep

refactor of the general runtime. `ARCHITECTURE.md` explicitly says the current goal is a stable smaller research repo before risky refactoring, and `DEVELOPMENT.md` says new JEPA work should stay behind explicit config or CLI gates while preserving the three baseline paths. ¹⁹

The following diagram captures the cleanest module relationship for that approach. It is a synthesis of the repo's current runtime surface and the JEPA additions the docs already point toward. ²⁰

```
flowchart LR
    A[VisualDomainAdapter and dataset loaders] --> B[RetinaAdapter and spike encoding]
    B --> C[Stage-1 branch and hemisphere states]
    C --> D[Context latent head]
    C --> E[EMA target latent head]
    D --> F[Predictor head]
    E --> G[JEPA latent loss]
    F --> G
    C --> H[Existing classifier and fusion readouts]
    G --> I[Latent replay and plasticity hooks]
    H --> J[Protected baseline comparison]
    G --> K[Stage-wise separability diagnostics]
```

Gaps and Technical Challenges

The central technical mismatch is that I-JEPA learns by predicting **representations of masked regions from visible context**, while SNNJepa's retained benchmark is organized around retinal branches, hemispheres, fixations, and classifier/fusion states rather than dense patch-token transformers. In I-JEPA, target blocks are formed from a target encoder's output, context is a sparse but informative visible block, prediction happens in latent space, and the target encoder is stabilized by EMA; the repo, by contrast, currently documents stage-1 activation patterns, bilateral fusion, and a future desire to add context/predictor and target states on top of the spiking sensory rollout. The implementation question is therefore not "how do we port ViT I-JEPA?" but "what should count as a predictive token in this spiking system?" The most plausible answer, given the current code surface, is branch-level, fixation-level, or hemisphere-level latent summaries, not raw neuron states or pixel patches. ²¹

That choice makes spike and timing encoding the first real challenge. The repository exposes multiple encoding abstractions—rate, temporal, and population encoders—and its adapter README explicitly frames sensory adapters as turning external data into spike trains. But the protected mainline is not a general-purpose sequence pretrainer; it is a retina-classification system whose identifiable wins come from usable upstream codes, and whose failures often came from changing structure or downstream logic without improving separability. A JEPA layer that sits too close to sparse, unstable, fixation-sensitive spike events will inherit exactly that fragility. A JEPA layer that sits too late, after the classifier has already collapsed ambiguity, will no longer be learning a useful world-model-like code. That is why the recommended landing zone is the **stage-1 or immediate post-stage-1 representation boundary**. ²²

Masking is the second major challenge. I-JEPA's masking works because the targets are large enough to be semantic and the visible context is sparse but spatially informative. SNNJepa's docs already point toward "masked-view or withheld-fixation targets" and "temporal consistency across fixations," which is important: the repository has already spent a great deal of effort on static front-end retuning, branch reshaping, figure-ground add-ons, and other same-surface manipulations that did not yield promotable gains. The masking policy therefore should not be another small perturbation of the current classifier surface. It should create a **meaningfully different predictive task**, such as withholding a fixation, an entire branch group, or a hemisphere-local latent slice, while asking a context state to predict the missing latent. That is squarely in line with both the repo roadmap and the JEPA literature. ²³

Predictor design and loss design are the third challenge. JEPA prediction is not reconstruction. The predictor should be narrow, conditioned on position or target identity, and trained against a more stable target representation than the context branch itself. I-JEPA is explicit that JEPA collapse remains a concern and that asymmetric design with an EMA target encoder is important; the broader JEPA framing also treats prediction in latent space as the core distinction from input reconstruction. For SNNJepa, that argues against a heavy decoder and in favor of a small predictor conditioned on fixation/branch position tokens, trained with normalized latent distance and an explicit anti-collapse safeguard. I infer that a variance-floor or covariance penalty is prudent here, not because the repo already has it, but because JEPA-style latent learning on sparse spike-derived states is especially vulnerable to trivial low-variance outputs. ²⁴

Training stability and runtime integration are the fourth challenge. The repo explicitly says it does not want to become a surrogate-gradient or ANN-conversion project, and `RabbitHoles.md` repeatedly warns against coupling stable benchmark behavior to immature experimental graph paths. That means the first JEPA trainer should be a **contained addition**: ideally a separate executable or a clearly gated path within the existing benchmark harness, with logging strong enough to prove the mechanism is active before any promotion attempt. The docs are also quite strict that local improvements are not enough: new work must show mechanistic validity, then stage-wise separability, then a protected benchmark win, and then confirmation if the win is narrow. This should shape both implementation order and promotion criteria. ²⁵

Finally, benchmarking is a first-class challenge, not an afterthought. The documented protected reference surfaces include unilateral retina static at 86.17%, bilateral retina static at 87.29%, bilateral retina continuous at 88.85% post-correction, and the CIFAR bilateral natural-features reference at 38.16% on the 1000/class, 5000-test benchmark. `NEXT_STEPS.md` and `RabbitHoles.md` are unambiguous: a new representation path that improves local metrics but does not clear the protected gate should be treated as a dead end, not as a near miss. Any JEPA plan that does not operationalize those gates will drift back into exactly the failure mode the repo is trying to eliminate. ²⁶

Recommended Architecture and Algorithms

The design principle from the literature is straightforward: a JEPA learns by making a context representation predictive of a target representation in latent space, not by reconstructing raw input, and not by demanding baked-in invariances from aggressive view augmentation. The design principle from the repo is equally straightforward: stay on the spiking substrate, preserve the current benchmark harness, and bias new work toward upstream representational improvement rather than smarter downstream choosing. The recommended architecture below is the intersection of those two principles. ²⁷

Design axis	Priority one	Priority two	Priority three	Rationale
Spiking encoder strategy	Freeze current retinal rollout initially; add JEPA heads above stage-1 states	Allow local plasticity in the latent readouts after the latent task is stable	Revisit deeper encoder adaptation only after gated wins	This keeps the protected baseline intact and isolates whether JEPA helps representation quality before the repo pays the cost of changing the sensory code.
JEPA tokenization	Fixation-level and branch-level latent tokens	Hemisphere-level summary tokens	Object-state tokens once object-state formation is demonstrably usable	These match the repo's current structure better than pixel patches and fit the roadmap's emphasis on partial observation, fixations, and object-centric recurrence.
Context/target split	Visible fixations or visible branch subset as context; withheld fixation or withheld branch subset as target	Cross-hemisphere target prediction	Temporal next-state prediction across settling cycles	This is the cleanest translation of I-JEPA masking into the repo's existing branch/fixation organization.
Predictor architecture	Small MLP with position/branch/fixation token conditioning	GRU or lightweight temporal aggregator over fixation sequence	Cross-attention predictor over object-state tokens	Start narrow and auditable. The repo's history argues against early architectural bloat.
Target encoder	EMA copy of context-latent head	Slow-moving target centroids plus EMA head	Object-state target bank with replay	EMA target encoders are central to stable JEPA training in I-JEPA.
Loss	Cosine or normalized MSE between predicted and target latents, plus variance floor	Smooth L1 on normalized latents	Add covariance decorrelation if collapse appears	Latent prediction should stay geometric and simple. Anti-collapse terms are a pragmatic safeguard.

Design axis	Priority one	Priority two	Priority three	Rationale
Replay / plasticity	Replay latent sequences, not classifier exemplars	Eligibility-trace-weighted latent consolidation	Object-state replay after base JEPA is stable	This directly matches the roadmap's "sequence replay as state replay."

The most important prioritization choice is **not** to make the existing classifiers smarter at the outset. The classification modules—HierarchicalKNN, MajorityVoting, WeightedDistance, WeightedSimilarity, and the staged fusion surfaces—should initially be treated as **evaluation probes** over the new latent states, exactly because the docs warn that downstream tuning on weak upstream codes is the dominant repeated failure mode. ²⁸

A clean first implementation would expose a JEPA sample constructor over the current sensory rollout. It should choose visible and hidden slices at the level of fixations and branches, create a context latent from the visible slices, create target latents from the hidden slices using an EMA head, and predict those target latents from the context latent plus location identity. That is much closer to the repo's roadmap than trying to force a ViT patch-mask abstraction directly onto spike trains. ²⁹

```
// Suggested new abstraction layer
struct JepaTargetSpec {
    int branch_id;
    int fixation_id;
    std::vector<double> target_latent;    // stop-gradient / EMA path
};

struct JepaSample {
    std::vector<double> context_latent;
    std::vector<JepaTargetSpec> targets;
};

JepaSample build_jepa_sample(const SensoryRollout& rollout,
                             const JepaMaskConfig& cfg) {
    auto visible_mask = sample_visible_context(rollout, cfg);    // no target
    overlap
    auto hidden_specs = sample_hidden_targets(rollout, visible_mask, cfg);

    auto context_state = aggregate_visible_states(rollout, visible_mask);
    auto z_ctx = context_head(context_state);

    std::vector<JepaTargetSpec> targets;
    for (const auto& spec : hidden_specs) {
        auto hidden_state = aggregate_hidden_state(rollout, spec);
        auto z_tgt = ema_target_head(hidden_state);    // stop-grad
    in training
        targets.push_back({spec.branch_id, spec.fixation_id, z_tgt});
    }
}
```



```

    return {z_ctx, std::move(targets)};
}

```

The corresponding training hook should be as small and inspectable as possible. The first version should log latent norms, pairwise cosine similarities, variance per dimension, and shuffled-target controls before it attempts any online-plasticity coupling. That order is dictated by the repo's mechanistic-validity gate. 30

```

for (const auto& image : training_stream) {
    SensoryRollout rollout = build_rollout_from_retina(image, config);

    JepaSample sample = build_jepa_sample(rollout, mask_cfg);

    double loss = 0.0;
    for (const auto& tgt : sample.targets) {
        auto position_code = encode_target_identity(tgt.branch_id,
tgt.fixation_id);
        auto z_pred = predictor(sample.context_latent, position_code);

        loss += cosine_distance(normalize(z_pred),
normalize(tgt.target_latent));
        loss += variance_floor_penalty(z_pred);
    }
    loss /= std::max<size_t>(1, sample.targets.size());

    optimizer.step(context_head, predictor, loss);
    ema_update(target_head, context_head, ema_decay);

    log_latent_metrics(sample, loss); // norm, variance, within-
image cosine
    log_separability_probe(sample, label); // centroid margin / class
separation
}

```

The literature also points to a broader conceptual lesson that matters in this repo: predictive learning in spiking systems has historically worked best when temporal credit assignment is made explicit and local state carries useful eligibility information. Foundational predictive-coding work by Rao and Ballard framed hierarchical predictions and residual errors as the core computational principle; Urbanczik and Senn showed a local rule minimizing somatodendritic prediction error; e-prop and DECOLLE showed that online or local-error learning in recurrent/deep spiking systems can work without standard BPTT assumptions. None of those papers is a drop-in JEPA recipe, but together they support the repo's strategic instinct to build a predictive latent layer on top of an event-driven substrate rather than abandoning the spiking regime.

31

Incremental Implementation Roadmap

The roadmap below is intentionally conservative, because the repo's docs are conservative for good reason. The objective is to add JEPA capability while preserving the benchmark surface that the repository treats as protected. ³²

Step	Goal	Files to add or modify	Validation checkpoints	Experiments	Effort
1	Add JEPA scaffolding without changing baseline behavior	Add: <code>include/snnfw/jepa/JepaConfig.h</code> , <code>JepaLoss.h</code> , <code>JepaSample.h</code> , <code>src/jepa/*</code> ; Modify: <code>src/CMakeLists.txt</code> , <code>experiments/CMakeLists.txt</code> , <code>docs/ARCHITECTURE.md</code> , <code>docs/JEPA_ROADMAP.md</code>	Baseline binaries still build and baseline scripts still run unchanged	No-learning smoke tests; compile-only plus baseline regression rerun	Low
2	Expose stable latent tap points above the current spiking rollout	Modify: <code>experiments/retina_classification.cpp</code> ; optionally add <code>include/snnfw/jepa/JepaStateExtractor.h</code> and <code>src/jepa/JepaStateExtractor.cpp</code>	Mechanistic validity: nonzero latent norms, deterministic dimensions, branch/fixation identity preserved in logs	MNIST and EMNIST latent dump runs; no training yet	Medium
3	Implement masking and context/target sample generation	Add: <code>JepaMaskSampler.*</code> ; Modify: benchmark harness to build visible/hidden splits from fixations/branches	No leakage between context and targets; target coverage histograms; shuffled controls worse than true pairs	Ablate withheld fixation vs withheld branch vs mixed masking	Medium

Step	Goal	Files to add or modify	Validation checkpoints	Experiments	Effort
4	Implement EMA target head, predictor, and latent loss	Add: <code>JepaHeads.*</code> , <code>JepaTrainer.*</code> ; Modify: experiment config parsing and CLI gates	Alignment loss decreases; variance floor stays healthy; no collapse to near-constant latents	MNIST pretraining smoke; EMNIST stability ablations; EMA vs no EMA; cosine vs normalized MSE	Medium
5	Use existing readouts strictly as probes of the new latent code	Modify: readout hookup in <code>retina_classification.cpp</code> ; optionally add <code>LatentProbeEvaluator.*</code>	Stage-wise separability: centroid margins, probe accuracy, same-image vs different-image cosine spread improve at the intended boundary	Frozen-latent probes across MNIST and EMNIST; compare raw stage-1 vs JEPa latent	Medium
6	Promote only if protected gates move; then add latent replay	Add: <code>JepaReplay.*</code> ; Modify: benchmark harness for latent-sequence replay and checkpointing	Protected benchmark win: beats appropriate reference; Confirmation: rerun any narrow gain	EMNIST protected-gate reruns; temporal consistency and replay on/off	High

Step	Goal	Files to add or modify	Validation checkpoints	Experiments	Effort
7	Bring JEPA to CIFAR only after earlier gates are clear	Modify: CIFAR config path and JEPA sampler; possibly add HDF5 artifact dumps using existing HighFive dependency	CIFAR branch/ hemisphere separability improves before headline accuracy is interpreted	CIFAR natural-features path: masking ratio sweep, predictor-width sweep, replay on/off, confirmation reruns	High

Two implementation choices deserve emphasis. First, step five is where the current classification surface becomes valuable—not as the learning target, but as a stable evaluation probe. That is exactly what the repo’s “fix the code, not the chooser” discipline implies. Second, step seven should not start early. `RabbitHoles.md` is effectively a long empirical argument that CIFAR can absorb huge amounts of complexity while teaching very little if the upstream code is still weak. ³⁰

The timeline below captures the intended gating logic. It is deliberately a decision flow, not a calendar promise, because `NEXT_STEPS.md` explicitly prioritizes stage-gated stopping over sunk-cost continuation. ³³

```

flowchart LR
  A[Scaffold JEPA modules] --> B[Expose latent tap points]
  B --> C[Add masking sampler]
  C --> D[Train EMA target plus predictor]
  D --> E[Probe stage-wise separability]
  E --> F{Protected gate win?}
  F -- No --> G[Stop, inspect instrumentation, revise representation]
  F -- Yes --> H[Add latent replay and temporal consistency]
  H --> I[Run confirmation reruns]
  I --> J[Promote to CIFAR stress test]

```

Testing and Evaluation Plan

The evaluation plan should be governed almost verbatim by `NEXT_STEPS.md`: mechanistic validity first, stage-wise separability second, protected benchmark win third, and confirmation fourth. That structure is materially better than a generic ML train/val/test split for this repo, because the rabbit-hole history shows that the main failure mode is not lack of local metric movement. It is misreading local metric movement as progress when the protected benchmark has not moved. ³⁰

Mechanistic validity for JEPA in this repo should mean concrete things. The latent path must be live, target masks must not leak visible evidence, latent dimensions must not collapse toward zero variance, and same-image predicted/target pairs must score better than shuffled or cross-image controls. This is the correct first gate because the rabbit-hole history repeatedly shows that the project has lost time on ideas whose effective pathways were not actually carrying useful signal. 26

Stage-wise separability should then be measured at the exact boundary the JEPA addition is supposed to improve. The repo already frames this correctly in prose: branch centroid accuracy, hemisphere purity, post-normalization margin, and fusion-proxy behavior are the right kinds of stage-local questions. For JEPA, I would extend that with latent-specific diagnostics: within-image context/target cosine, cross-image separation, centroid accuracy of frozen latent probes, latent norm histograms, variance-by-dimension, and collapse-target tracking. If the intended JEPA boundary does not become more separable, the path should stop there rather than migrating effort into classifier or fusion retuning. 34

Protected benchmark evaluation should use the repo’s documented reference surfaces exactly as the docs describe them. In practical order, that means: MNIST for fast smoke iteration, EMNIST as the first serious promotion target, and CIFAR-10 bilateral natural-features only after earlier gates succeed. The docs explicitly preserve these three dataset regimes for that purpose, and the rabbit-hole file lists the active reference metrics that new paths should beat rather than casually replace. 35

Because the repo has no visible top-level tests directory and the retained CMake surface does not show formal test registration, I would add a small but meaningful automated layer even before full JEPA promotion: deterministic mask-sampler tests, latent-shape tests, no-leakage tests, EMA-update tests, and one smoke regression on each protected dataset path. Those need not become a giant framework before useful work begins, but the current surface would materially benefit from at least a narrow correctness harness. The absence of that harness is one of the most important repo-level gaps for safe JEPA integration. 36

Risks, Mitigation, and Instrumentation

The main risks are not mysterious. They are exactly the risks the literature and the repo docs both point to: collapse, leakage, measuring the wrong stage, overcomplicating the harness, and mistaking small sampled wins for robust promotion. 37

Risk	Why it matters here	Mitigation
Latent collapse	JEPA-style prediction in embedding space can converge to low-information outputs if asymmetry is weak	EMA target head, narrow predictor, variance floor, shuffled-target control, latent variance dashboards
Mask leakage	With fixations and branches, it is easy to hide nominally “target” state while still leaking the same evidence through overlap	Explicit no-overlap sampler, visible/hidden provenance logging, leakage unit tests

Risk	Why it matters here	Mitigation
Wrong token granularity	Tokens that are too raw will be unstable; tokens that are too late will inherit classifier collapse	Start at stage-1 or immediate post-stage-1 summaries; compare fixation-, branch-, and hemisphere-level tokenizations
Benchmark contamination	Mixing immature experimental logic into the stable harness was already a documented rabbit hole	New executable or explicit JEPA config gates; baseline path must remain byte-for-byte runnable
Local metric mirage	The repo has already seen many paths improve local metrics without moving the protected gate	Promotion requires separability improvement and protected benchmark win and confirmation
CIFAR premature escalation	CIFAR has historically absorbed large complexity with little mainline value	Do not move to CIFAR until MNIST/EMNIST JEPA gates are clean

For instrumentation, I would make the reporting surface more JEPA-specific but keep it compatible with the repo's current diagnostics mindset. The most valuable charts are: latent variance by dimension over training; same-image predicted-vs-target cosine versus different-image cosine; stage-wise centroid accuracy from raw stage-1 to JEPA latent to fusion probe; mask coverage by branch and fixation; replay benefit versus no replay; and promotion dashboards that visibly align with the four decision gates in `NEXT_STEPS.md`. Since the top-level build already brings in `nlohmann/json`, HighFive, and libsonata, the easiest durable artifact format is HDF5 or JSON-based latent dumps rather than ad hoc console-only logging. ³⁸

One final point is worth stating plainly. The repo's own documents already contain the strategic answer to "what kind of JEPA should this become?" The answer is **not** a generic masked-image model retrofitted onto spikes, and **not** a downstream classifier hack wearing JEPA terminology. It is a spiking, fixation-aware, branch-aware latent predictive model introduced at the right boundary, instrumented hard enough to prove it is alive, and promoted only when it creates a better code before it asks the chooser to be smarter. That is the recommendation most consistent with the retained code surface, the documented dead ends, and the primary JEPA literature. ³⁹

¹ ¹⁴ ¹⁵ ²⁹ https://raw.githubusercontent.com/deanhorak/SNNJepa/main/docs/JEPA_ROADMAP.md
https://raw.githubusercontent.com/deanhorak/SNNJepa/main/docs/JEPA_ROADMAP.md

² ⁶ ¹⁸ ³⁶ <https://github.com/deanhorak/SNNJepa>
<https://github.com/deanhorak/SNNJepa>

³ ⁸ ¹⁰ ¹¹ ¹⁹ ²⁰ <https://raw.githubusercontent.com/deanhorak/SNNJepa/main/docs/ARCHITECTURE.md>
<https://raw.githubusercontent.com/deanhorak/SNNJepa/main/docs/ARCHITECTURE.md>

⁴ ⁵ ³⁰ ³² ³³ ³⁴ ³⁸ https://raw.githubusercontent.com/deanhorak/SNNJepa/main/docs/NEXT_STEPS.md
https://raw.githubusercontent.com/deanhorak/SNNJepa/main/docs/NEXT_STEPS.md

⁷ <https://openreview.net/pdf?id=BZ5a1r-kVsf>
<https://openreview.net/pdf?id=BZ5a1r-kVsf>

9 16 17 23 25 39 <https://raw.githubusercontent.com/deanhorak/SNNJepa/main/README.md>
<https://raw.githubusercontent.com/deanhorak/SNNJepa/main/README.md>

12 28 <https://github.com/deanhorak/SNNJepa/tree/main/src/classification>
<https://github.com/deanhorak/SNNJepa/tree/main/src/classification>

13 <https://github.com/deanhorak/SNNJepa/tree/main/src/learning>
<https://github.com/deanhorak/SNNJepa/tree/main/src/learning>

21 24 27 37 <https://arxiv.org/abs/2301.08243>
<https://arxiv.org/abs/2301.08243>

22 <https://github.com/deanhorak/SNNJepa/tree/main/include/snnfw/adapters>
<https://github.com/deanhorak/SNNJepa/tree/main/include/snnfw/adapters>

26 <https://raw.githubusercontent.com/deanhorak/SNNJepa/main/docs/RabbitHoles.md>
<https://raw.githubusercontent.com/deanhorak/SNNJepa/main/docs/RabbitHoles.md>

31 https://www.nature.com/articles/nn0199_79
https://www.nature.com/articles/nn0199_79

35 <https://raw.githubusercontent.com/deanhorak/SNNJepa/main/docs/BASELINES.md>
<https://raw.githubusercontent.com/deanhorak/SNNJepa/main/docs/BASELINES.md>